ARI: An Autonomous Research Infrastructure

Takanori Kotama

Snapshot v0.7.1, 2026

Abstract

We describe **ARI**, an autonomous research infrastructure that couples a tree-search-based exploration loop with a paper-generation pipeline. The exploration loop is a best-first tree search (BFTS) over research-step nodes; expansion uses a ReAct-style agent that issues tool calls into a registry of fourteen domain skills. The paper pipeline turns the surviving lineage into a draft, then iteratively refines it under a hierarchical rubric. Every artefact lives inside a single checkpoint directory, which is the unit of reproducibility, lineage, and cost accounting. This report formalises the algorithms, describes the implementation in source-code terms, presents an evaluation snapshot, and discusses the determinism / novelty trade-off that constrains the design.

1 Introduction

1.1 Motivation

Research workflows in machine learning and adjacent fields share a common skeleton: brainstorm a research idea, design and run an experiment, analyse the result, write a paper, and iterate. Each step is increasingly tractable for large language models, but stitching them into a single autonomous loop has, so far, been an open engineering problem. The published systems that come closest — AIDE, the AI Scientist family, VirSci, and PaperBench evaluators — each fix one or two of these axes while leaving the others manual.

ARI exists to fill that gap. We assume the user supplies a research question and a compute budget; the system returns a publishable paper, the experimental code that backs every claim, and a complete audit trail of how it got there. The audit trail is non-negotiable: every node of the search tree, every tool call, every model invocation, and every dollar of API spend is recorded inside a single *checkpoint directory* ([2] adopt a similar discipline). The checkpoint is the unit of reproducibility.

1.2 Contributions

This report formalises three contributions:

1. A **best-first tree search** over research-step nodes whose selection score $U(n)$ separates the empirical reward, a novelty term, and a depth penalty (section 3.1). The implementation lives in `ari-core/ari/orchestrator/bfts.py:114`.
2. A **paper-generation pipeline** keyed off a hierarchical rubric R whose leaf judges grade individual claims; the system score is $\text{score}(P) = \sum_i w_i \, \text{judge}_i(P)$ (section 4.2). The PaperBench replicator is integrated as a tool call (section 4.4).
3. A **checkpoint scoping** discipline that makes the pipeline fully reproducible: every external state (memory, cost ledger, lineage pointers) is written below `$ARI_CHECKPOINT_DIR`, never the user's home directory. This is the operational consequence of the P2 determinism principle (section 2.5).

1.3 Scope of this report

We freeze the snapshot at version v0.7.1 of `ari-core` and the fourteen `ari-skill-*` packages. Implementation citations name file and line; readers can verify them against the repository in the section 2.1 layout map. The numerical results that this version of the report would normally place in section 5 are not yet available — the frozen checkpoint that they would draw from has not been produced — and that chapter is therefore marked "deferred". The snapshot version is part of the title page so that later code modifications do not retroactively invalidate any claim the report does make.

1.4 Organisation

Section 2 gives a top-down view of ARI: the orchestrator, the fourteen skills, the pipeline DAG, and the design principles. Sections 3 and 4 are the two technical cores, each with its own formal definitions and empirical observations. Section 5 is the evaluation snapshot; section 6 positions the work; section 7 collects open problems.

2 System Overview

2.1 The full stack

Figure 1 draws the full ARI stack at v0.7.1. The system splits into one driver and many tool providers. The driver (`ari-core`) owns the orchestrator, the BFTS engine, the lineage walker, the agent loop, the checkpoint, the cost tracker, and the configuration loader. Each skill is a separate Python package

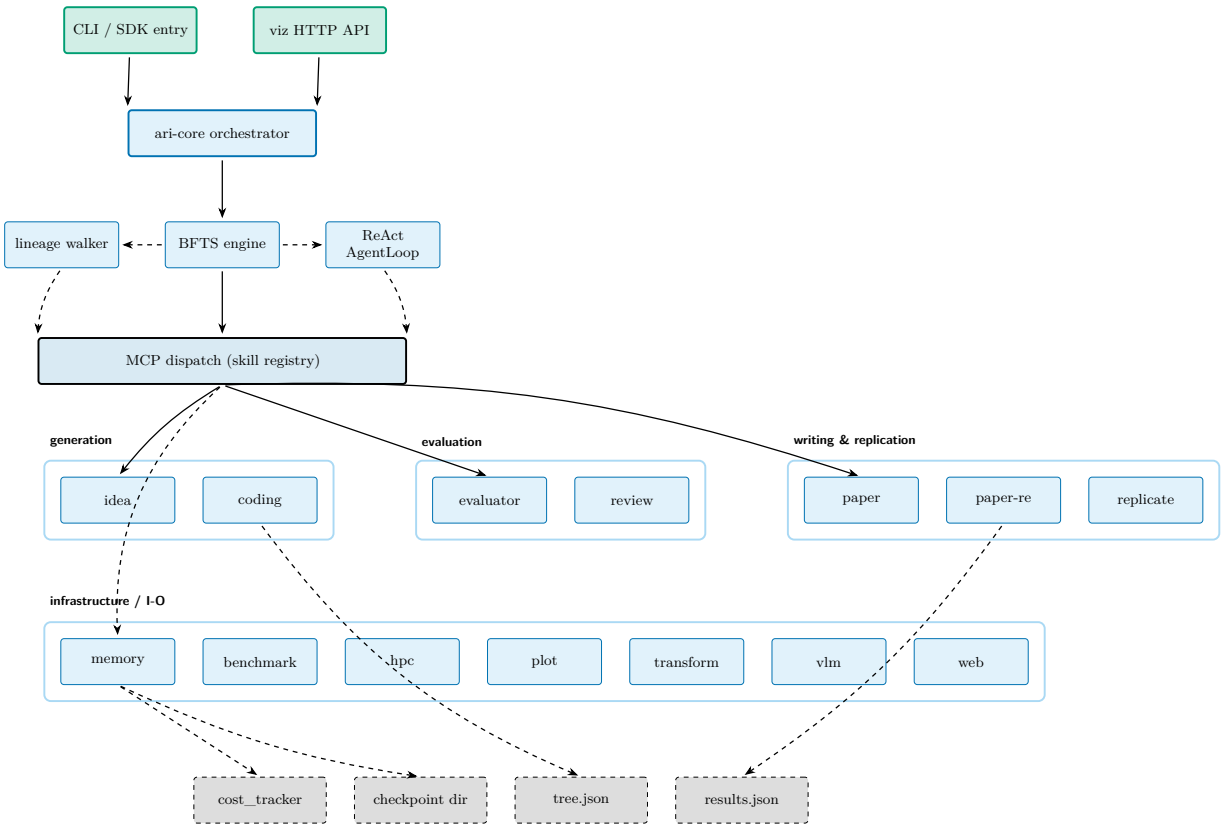

Figure 1: The full ARI stack at v0.7.1. The CLI / SDK entry-points drive a single orchestrator (`ari-core`); the orchestrator owns three engines (BFTS, lineage walker, ReAct `AgentLoop`) and dispatches into the 14-skill registry through the MCP layer. All persistence lands inside one checkpoint directory: cost ledger, `tree.json`, and `results.json`. Solid edges are control; dashed edges are state.

shipped as an MCP server with its own `skill.yaml` declaration. The fourteen skills currently in tree are: `benchmark`, `coding`, `evaluator`, `hpc`, `idea`, `memory`, `orchestrator`, `paper`, `paper-re`, `plot`, `replicate`, `transform`, `vlm`, and `web`.

The orchestrator dispatches calls but does not perform domain work itself. This separation matters because skills can be added or swapped without touching the search algorithm: the BFTS code in `ari-core/ari/orchestrator/bfts.py:114` is unaware of which skills participate; everything is mediated by the MCP dispatch layer.

The smaller diagram in fig. 2 below is the "operational" view (control + state outputs) used in the rest of the report; fig. 1 is the "layered" view (driver MCP skills persistence).

2.2 Pipeline DAG

A run reduces to a four-station pipeline (fig. 3). The idea station selects or generates research questions; the exploration station spawns nodes under BFTS; the paper station compiles the best lineage into a draft; and the review station grades the draft against the rubric. The graph is a DAG plus one back edge from review to paper: only this edge can introduce non-monotonicity (refining worsens an axis), and the *convergence criterion* (section 4.3) is what bounds the back-edge count.

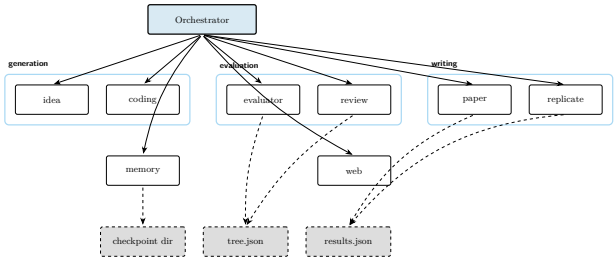

Figure 2: Operational view: orchestrator dispatches into the skill registry; every skill writes only into the active checkpoint directory. Dashed edges indicate state.

The pipeline runner is `ari-core/ari/pipeline/orchestrator.py:131` (`run_pipeline`); the scientific-data assembler that closes the gap between exploration output and paper input is `build_scientific_data` on line 68 of the same file.

2.3 BFTS control flow

Figure 4 traces a single BFTS step end-to-end. The implementation is laid out in section 3; the diagram fixes the vocabulary used by the rest of the report (frontier, selection, fallback, expand, prune, stagnation) and the prompt files (`bfts_select.md`, `bfts_expand_select.md`, `bfts_expand.md`) that materialise each LLM-driven step.

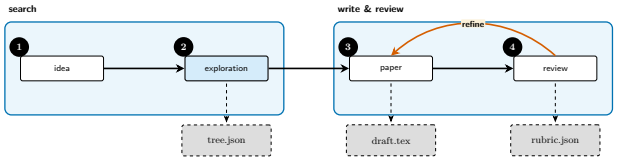

Figure 3: Pipeline DAG: idea → exploration → paper → review, with *refine* closing the back edge. Dashed outputs are persisted to the checkpoint; the refine cycle rewrites the draft and re-runs the review.

2.4 paper-re component view

Figure 5 is the second-level view of `paper-re`. ARI's contribution at this layer is small but specific: the `AriPBSolver` subclass overrides only `_get_tools` (to insert `_BoundedOutputTool`) and `_run_agent` (to bypass docker sanity-check and adapt vendor paths); everything else flows through PaperBench upstream verbatim, which keeps ARI aligned with the upstream replication-task framing. Section 4 walks the diagram in detail.

2.5 Checkpoint scoping and the design principles

ARI is built around five *design principles* (P1–P5). The binding one is **P2: determinism**. Every randomised step records its seed; every LLM call records its model identity, prompt, and output; every tool call carries a `cow_node_id` so its writes land inside the right node's working tree. Re-running a checkpoint must reproduce the same artefacts byte-for-byte, except for fields explicitly marked as wallclock or as model-side non-determinism.

The other principles, in summary:

- **P1 single artefact tree**: every output of a run lives below `$ARI_CHECKPOINT_DIR`. Nothing leaks to `$HOME` or `/tmp`.
- **P3 small components**: each skill is a separate package; replacing one skill does not require rebuilding `ari-core`.
- **P4 explicit lineage**: every node and every checkpoint record their parent, so a derivative experiment can recompute only its delta.
- **P5 cost transparency**: `cost_tracker.py` attaches dollar costs to every model call so the user can bound a run.

The checkpoint serialiser writes three top-level files. The full search tree goes to `tree.json` via `checkpoint.py:49`; per-node payloads go to `nodes_tree.json` via `checkpoint.py:59`; and end-of-run aggregates go to `results.json` via `checkpoint.py:64`. Lineage walks across checkpoints use `lineage.py:126` (`walk_ancestor_ckpts`) to climb from a child to its ancestors.

Figure 6 draws the on-disk shape of a checkpoint: shared run-level files on the left, per-node working trees on the right. The shape is the contract: any tool that takes ARI's output as input (a follow-up run, a regression check, a paper draft) points at a single such directory.

3 Exploration Algorithm

3.1 Formalisation

The exploration loop maintains a tree $\mathcal{T} = (\mathcal{N}, E)$ in which each node $n \in \mathcal{N}$ represents a single research step: an idea revision, a code change, a benchmark run, an analysis, etc. A node carries

- a discrete state $s(n) \in \{\text{open}, \text{eval}, \text{done}, \text{failed}\}$ (`node.py:10`),
- a categorical *label* drawn from `NodeLabel` (`node.py:18`),
- per-axis evaluation metrics in a freeform dict `node.metrics`, including a distinguished scalar `_scientific_score` $\in [0, 1]$,
- a Boolean `has_real_data` that flags whether the node carries actual measurements rather than provisional text, and
- bookkeeping fields `depth` and `retry_count` used by the selection prompts.

The aggregation of axis-level signals into the scientific score is delegated to any implementation of the `Evaluator` protocol (`ari-core/ari/protocols/evaluator.py:19`); ARI does not pin a fixed linear combination. The protocol is the only seam between BFTS and downstream rubric evaluation, which keeps the search engine agnostic about how axes get reduced to a scalar.

The hard exploration budget is a single constant `config.max_total_nodes`, enforced by `bfts.py:should_prune` (line 245). Step / cost budgets are enforced separately by `cost_tracker.py:77` at the call site, not inside the BFTS engine.

3.2 Node selection (LLM-driven)

ARI deliberately does *not* reduce node ranking to a closed-form scalar utility. Instead, both `select_next_node` (line 167, "which node should the next agent step operate on?") and `select_best_to_expand` (line 258, "which completed node is most worth expanding?") hand a list of candidate descriptions to the LLM and parse a single 0-based index back. The candidate description for each node n is a one-liner of the form

```
[i] id=..., depth=..., retry=...,
has_real_data=..., metrics={...},
summary=..., diversity_bonus=+0.05
```

and the prompt that consumes the list lives at `ari/prompts/orchestrator/bfts_select.md`

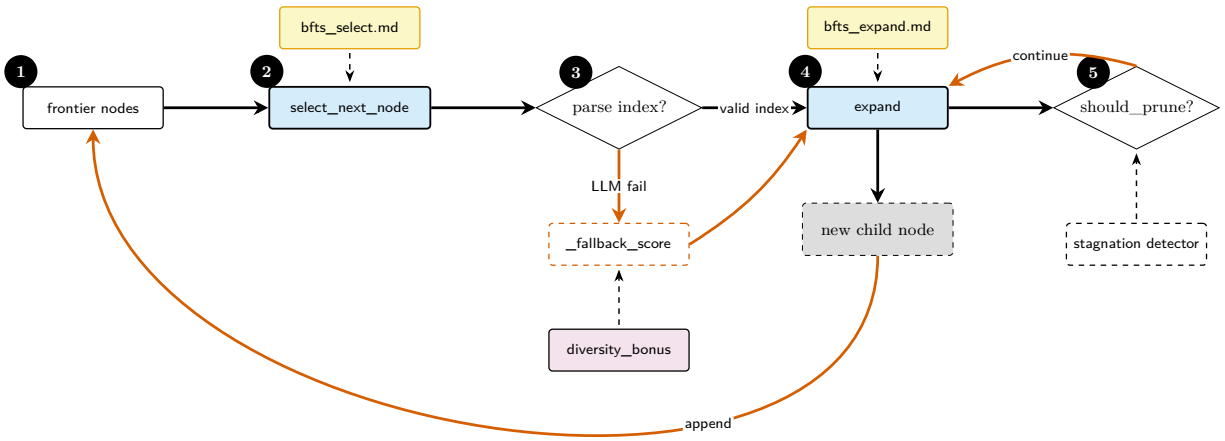

Figure 4: BFTS control flow. Two LLM-driven decision points (`select_next_node`, `expand`) each consult a separate prompt file; the selection decision falls back to a deterministic `_fallback_score` (which combines `_scientific_score` with `diversity_bonus`) when the LLM returns an unparseable index. Pruning is gated by both `should_prune` (max-total-nodes hard cut) and the stagnation detector. Detail in section 3.

(selection) and `ari/prompts/orchestrator/bfts_expand_select.md` (expansion target). The selection criteria the prompt enumerates are: nodes with `has_real_data=True` and strong metrics are high-value; unexplored directions are preferred over already-tried paths; metrics are weighed holistically (multi-objective); deeper nodes with excellent results are worth continuing; the `diversity_bonus` is treated as a soft tiebreaker only.

Diversity bonus

`BFTS.diversity_bonus` (line 142) tracks recent labels in `_recent_label_history` (a sliding window populated by `record_run`). For a node whose label is *under*- represented relative to the most common label in that window, the function returns

$$\mathrm{div}(n) = \begin{cases} +0.05 & \text{if } \mathrm{count}(\mathrm{label}(n)) \le \tfrac{1}{2} \max_{\ell} \mathrm{count}(\ell), \\ 0 & \text{otherwise.} \end{cases} \tag{1}$$

The constant 0.05 is small by design: scientific score still dominates ranking, and the bonus only breaks near-ties.

LLM fallback

If the LLM returns an unparseable index, `select_next_node` falls back to a deterministic ranking (`bfts.py:237`, `_fallback_score`):

$$\mathrm{\_fallback\_score}(n) = \texttt{\_scientific\_score}(n) + \mathrm{div}(n). \tag{2}$$

Nodes with `has_real_data=True` are preferred when any are available, and the candidate with the highest fallback score is returned. This guarantees the loop always makes progress even when the LLM call degrades.

3.3 Expansion (one child per call)

`bfts.py:expand` (line 315) generates *exactly one* new child per invocation. Callers wanting more children for the same parent call `expand` repeatedly, threading the previously created children through the `existing_children` argument so the LLM can avoid proposing duplicate directions.

The label of each new child is decided by the LLM rather than by a hardcoded template; the prompt (`ari/prompts/orchestrator/bfts_expand.md`) is fed:

- the parent's status (`succeeded` / `failed/no-real-data`) and `_scientific_score`;
- the parent's structured `node_report` (`delta_vs_parent` / `concerns` / `hints`, see `node_report/builder.py`) when present, so the planner can target specific weaknesses;
- sibling scores at the same depth, and ancestor scores from root to parent;
- tree-wide diversity metrics (unique labels seen so far, depth distribution); and
- the labels of children already spawned at this parent, so duplicates are not proposed.

A child is created in state open, handed to the agent loop for execution (section 3.6), and transitions to done when all axes are populated or to failed if the agent loop raises.

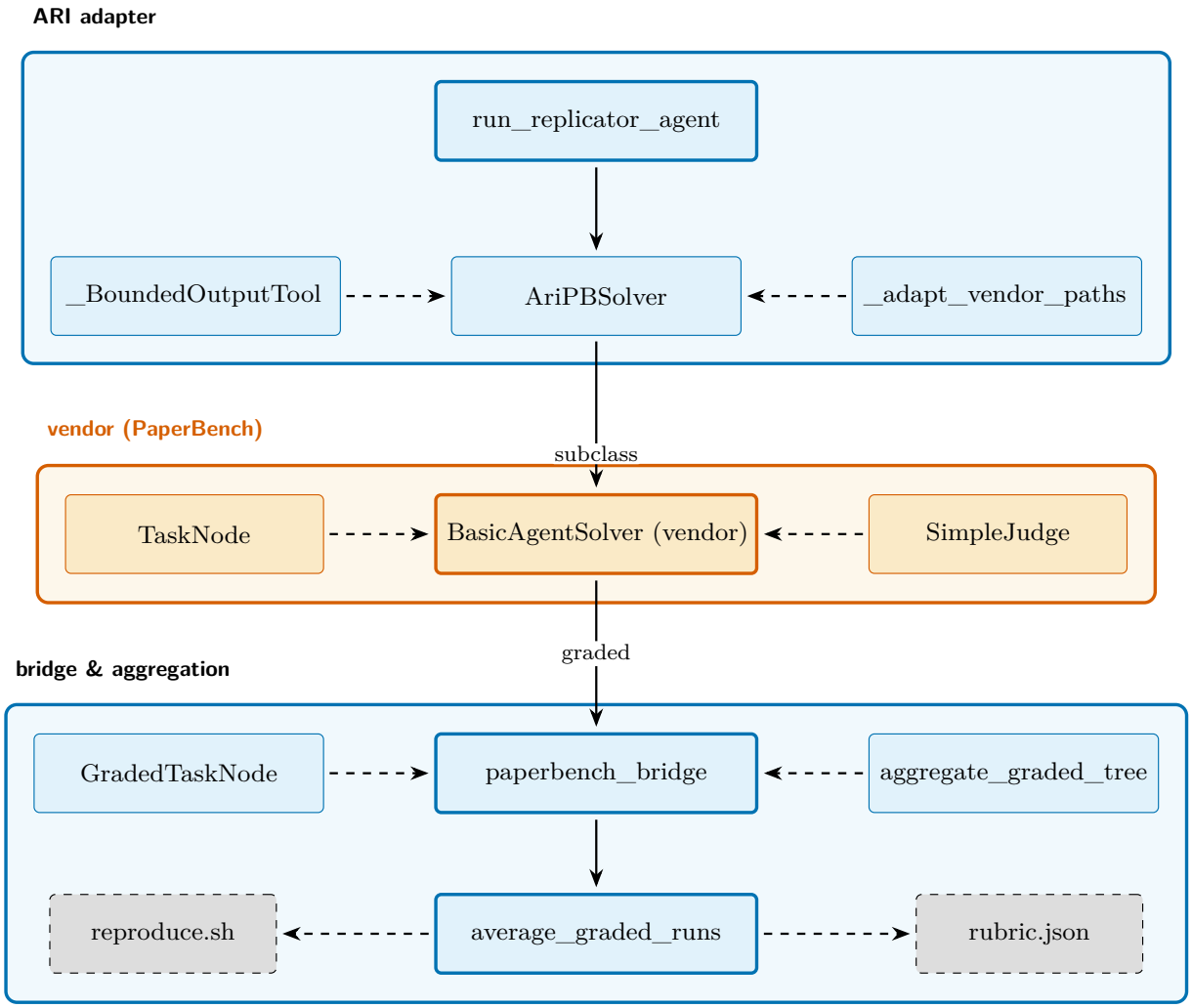

Figure 5: The `paper-re` skill's components. The top row is ARI's adapter layer (driver + solver subclass + bounded tool wrapper + path adapter); the middle row is the vendored PaperBench package (`BasicAgentSolver`, `TaskNode`, `SimpleJudge`); the bottom row is the bridge that collapses a per-submission `GradedTaskNode` tree to a single `rubric.json` score plus an ARI-side `reproduce.sh`. Detail in section 4.4.

3.4 Pruning and termination

`should_prune` (line 245) applies only *hard* cutoffs: when `self.total_nodes >= self.config.max_total_nodes`, further expansion stops. `depth` is intentionally treated as a *soft* signal that flows into the LLM prompt rather than a hard cap, so high-quality nodes appearing deep in the tree are not discarded mechanically.

The loop terminates when any of:

- the global `max_total_nodes` cap is hit;
- the per-call step / cost budgets enforced by `cost_tracker.py` run out;
- the *stagnation detector* (`lineage_decision.py:334`, `detect_stagnation`) observes that the running max of `_scientific_score` has not improved over a sliding window;
- a `LineageDecision` (`lineage_decision.py:149`) explicitly halts the branch.

Which of these fires most often is an empirical question that section 5 is meant to answer once a frozen checkpoint is in place; the present report does not claim a distribution.

3.5 Lineage and reproducibility

A lineage is the chain of ancestors of the best node, written into the checkpoint as the `lineage` key in `tree.json`. The `LineageState` dataclass (`lineage_decision.py:64`) captures the running statistics that BFTS uses to decide whether to continue, and the cross-checkpoint walker `lineage.py:126` (`walk_ancestor_ckpts`) provides parent pointers across runs. Idea pools are inherited via `lineage.py:157` (`get_idea_pool_for_ckpt`), and a ranked root-idea selection is recorded by `root_idea_selector.py:39` (`RootChoice`) so the choice is auditable post hoc.

Reproducibility hinges on three invariants. (i) Every randomised operation seeds itself from the node identifier. (ii) Every tool call records its arguments and out-

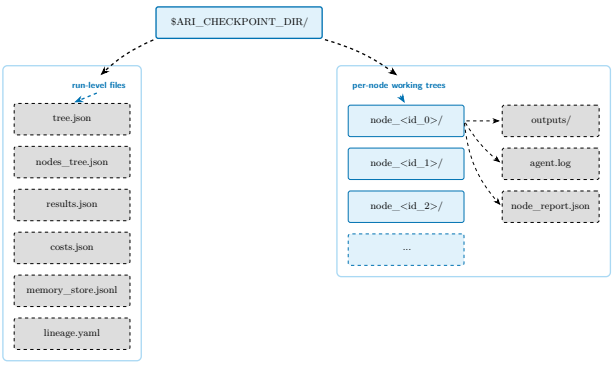

Figure 6: Checkpoint directory layout. Run-level files (left band) are written by the orchestrator; per-node working trees (right band) hold the artefacts produced inside one BFTS expansion. Reproducing a run is exactly "re-execute against this directory", and the layout guarantees that no other state is needed.

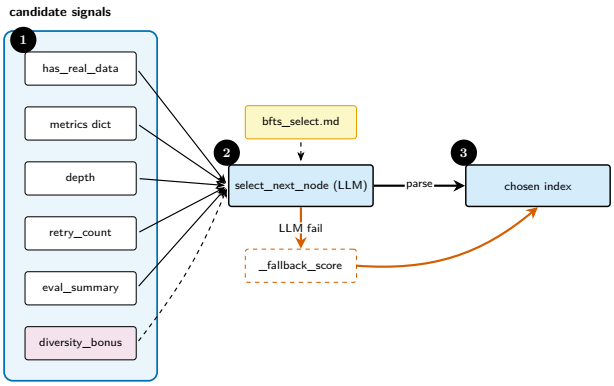

Figure 7: Selection signals fed to the LLM that picks the next node to operate on. Inputs are concatenated into a structured candidate description; the soft `diversity_bonus` is a tiebreaker, not a primary signal.

puts into the node's working tree, never the parent's — this is enforced by the MCP layer's `cow_node_id` check. (iii) The cost ledger (`cost_tracker.py:181`, `init`) attaches a per-node dollar amount, so the budget check is deterministic given the same prompts.

3.6 ReAct driver

Each node's expansion runs a ReAct-style agent loop (`ari-core/ari/agent/loop.py:77`, `class AgentLoop`). The maximum step count is `MAX_REACT_STEPS = 80` (`loop.py:25`); it can be overridden per-instance via the `max_react_steps` keyword. A separate guard `MIN_TOOL_CALLS = 2` (line 26) prevents trivially short trajectories.

The set of tools available to the agent is filtered per phase by `ari.agent.tool_manager`; for example, the exploration phase masks the paper-writing tools so that BFTS expansion cannot short-circuit into a draft. A small set of MCP tools is reserved for `ari-core` itself (memory CoW operations) and is hidden from the LLM, ensuring that the model cannot set an arbitrary node id and bypass the memory skill's copy-on-write check (see the docstring of `loop.py`).

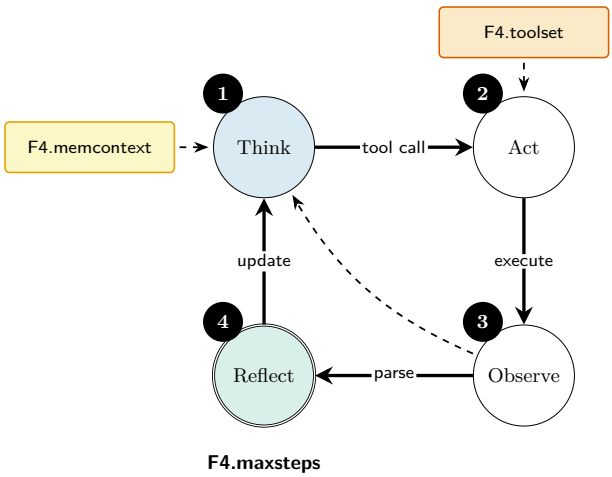

Figure 8: The ReAct loop. Solid edges are the main cycle; the dashed back edge represents an early-exit when the observation already closes the open question.

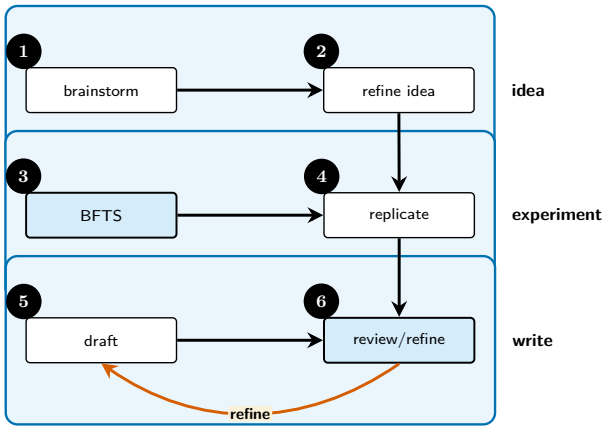

Figure 9: Paper-generation swim lane: idea, experiment, write. Each lane corresponds to a phase; the back edge from `review/refine` to `draft` is the only non-monotonic transition (section 4.3).

4 Paper Generation Pipeline

4.1 Pipeline overview

The paper-generation pipeline is the second tree of activity in ARI (fig. 9). Once exploration terminates, the surviving lineage is handed to the *paper* skill, which produces a draft, and the *review* skill, which scores the draft. The implementation is `run_pipeline` in `ari-core/ari/pipeline/orchestrator.py:131`, and the `build_scientific_data` helper on line 68 of the same file collates the per-node artefacts the paper skill needs.

A run produces three persisted artefacts: a `draft.tex` (the LaTeX paper proper), a `review.json` (per-leaf rubric scores and rationale), and a `rubric.json` (the rubric instance used to score this run, useful when the rubric is generated on-the-fly).

4.2 Rubric formalism

We model the *rubric* R as a finite tree whose leaves carry tuples $(c_i, w_i, \text{judge}_i)$. Each leaf i has a category descriptor c_i, a non-negative weight w_i with $\sum_i w_i = 1$, and a *judge function* $\text{judge}_i : P \to [0, 1]$ that returns a graded score. The paper score is the convex combination

$$\text{score}(P) \;=\; \sum_i w_i \,\text{judge}_i(P). \tag{3}$$

This formalism is shared with the PaperBench rubric tree, which ARI reuses directly through a vendored package (section 4.4): the leaf type `TaskNode` comes from `paperbench.rubric.tasks`, the graded leaf type `GradedTaskNode` from `paperbench.judge.graded_task_node`, and the weighted aggregation envelope from `aggregate_graded_tree` in `ari-skill-paper-re/src/_paperbench_bridge.py:75`.

Two grader families are used in production: an LLM-backed grader, implemented for ARI by wrapping PaperBench's `SimpleJudge` (`ari-skill-paper-re/src/_paperbench_bridge.py:54`); and a deterministic grader for items with a binary or numeric verifier (e.g. "reproducibility", "cost reported in absolute dollars"). Both implement the `Evaluator` protocol (`ari-core/ari/protocols/evaluator.py:19`).

4.3 Review/refine loop

The refine cycle iteratively rewrites P_t to P_{t+1} under feedback $J(P_t)$:

$$P_{t+1} \;=\; \text{Refine}\big(P_t, J(P_t)\big). \tag{4}$$

The cycle stops when the *convergence criterion* $\text{score}(P_{t+1}) - \text{score}(P_t) < \epsilon$ holds for two successive iterations, or when a per-pipeline cap of three iterations is reached. The empirical frequency at which each branch fires — convergence vs. cap — is what section 5 is meant to report once a frozen checkpoint is in place; we make no quantitative claim here.

The refine step is intentionally non-monotonic per axis: a fix to a clarity issue may slightly degrade a numeric axis. This is the source of the back edge in fig. 9; the convergence criterion is on the aggregate score, not per-axis.

4.4 PaperBench integration via `paper-re`

The *paper-re* skill (`ari-skill-paper-re`) is ARI's PaperBench-compatible replication driver. The integration is *not* a re-implementation: PaperBench is shipped vendored at `vendor/paperbench` so that ARI tracks upstream task and rubric semantics verbatim (the layout requirement is documented in PaperBench's `rubric.md §8`). Three integration points deserve mention.

1. Solver subclass: `AriPBSolver`

`ari-skill-paper-re/src/_replicator_agent.py:269` subclasses PaperBench's upstream `paperbench.solvers.basic.BasicAgentSolver`. The chz immutable-config rule requires the decorator on the subclass even when no fields are added; ARI's overrides are minimal:

- `_get_tools()`: every non-submit tool is wrapped in `_BoundedOutputTool` so that bash / python / file-read / search outputs are size-capped before they flow into the next API call. `submit` is dispatched by name and never has its `execute()` invoked, so it is passed through unwrapped.
- `_run_agent()`: bypasses `paperbench.solvers.utils.sanity_check_docker` via `_bypass_docker_sanity_check`, and rewrites the upstream hardcoded `/home/{submission,paper}` paths to ARI's workspace-relative layout (`LocalComputer` cwd = `repro_sandbox/`, with paper at `paper/` and submission at `submission/`). Prompt content otherwise comes verbatim from upstream's `get_instructions` / `get_system_message`, keeping ARI aligned with the upstream replication-task framing (7-day budget, "do not stop until you have replicated all results", etc.).

`AriPBSolver` also retains upstream's `check_for_existing_run` so a partially completed run can be resumed idempotently.

2. Rubric-driven artefact list

Vendor PaperBench is unaware of ARI's per-paper rubric tree, so `AriPBSolver._run_agent` appends an `EXPECTED_ARTIFACTS` block to the user instructions when `LocalPBTask.rubric_expected_artifacts` is non-empty. The appended block lists the paths a successful `reproduce.sh` must produce; without it, the grader's leaf claims would have no hint about which files to verify. The rubric tree itself is constructed by `ari-skill-paper-re/src/_paperbench_bridge.py:63` (`task_node_from_dict`).

3. Driver: `run_replicator_agent`

`ari-skill-paper-re/src/_replicator_agent.py:365` is the async entry point. It returns the standard `build_reproduce_sh` envelope {`populated`, `output_dir`, `files`, `expected_artifacts`, `max_runtime_sec`, `model`, `prompt_sha256`, `notes`, `warnings`}. The `output_dir` is both the agent's workspace and where ARI expects `reproduce.sh` after the rollout; the upstream solver writes `agent.log` and other bookkeeping into `run_dir`, which we point at the workspace itself so artefacts live alongside the reproduce script.

The default completer is `OpenAIResponsesTurnCompleterConfig` with model `gpt-5-mini` (overridable via the

ARI_MODEL_REPLICATOR environment variable); LiteLLM-routed alternatives are wired through the helpers in _litellm_completer.py. The default time limit is twelve hours per run; sandbox_kind is auto, falling back to a local Apptainer image when one is supplied.

4. Grading and aggregation

After the replication agent completes, grading is delegated through judge_submission in ari-skill-paper-re/src/_paperbench_bridge.py. The function adapts ARI's (paper_md, submission_dir, reproduce_log, judge_model) tuple to upstream SimpleJudge, which produces a GradedTaskNode tree per submission. Aggregation across runs is then handled by two helpers:

- aggregate_graded_tree (line 75): collapses one GradedTaskNode into a weighted scalar via the rubric weights w_i, plus per-category breakdown.
- average_graded_runs (line 91 region): takes the per-run mean over n GradedTaskNode trees, so a single replication score is reported even when the agent was rerun for noise reduction.

The two-stage design — vendor's SimpleJudge for leaf grading, ARI's adapters for shape and aggregation — means that upgrading PaperBench upstream is a vendor swap (no ARI code changes), while ARI retains full control of how leaf grades are combined into a single number. The same envelope feeds into eq. (3) as $\text{judge}_i \in \{0, 1\}$ when the leaf is binary.

4.5 Failure modes and guardrails

Three failure modes are common enough to be named:

- **LLM-only support**: the paper makes a claim that no node in the lineage substantiates. The mitigation is a static check: every numerical or asserted-causal claim must be traceable via a \code{...} reference to a node artefact.
- **Citation hallucination**: the paper cites a non-existent work. The mitigation is structural; we forbid the LLM from inventing citation keys and require all bib entries to be verified against Semantic Scholar (section 6).
- **Refine drift**: the score climbs but the paper loses coherence. The mitigation is the per-axis floor in the rubric: any axis falling below 0.5 aborts the refine.

Source-file selection at draft time uses node_selection.py:204 (select_source_files_for_publication); this is what guards against the LLM-only-support failure by enumerating exactly which lineage nodes a claim can lean on. The companion load_selected_sources (line 258) reads bytes for every (node_id, rel_path) pair and respects an optional size_budget so the paper draft never exceeds context.

4.6 Tool-output bounding for the replicator

The replicator agent can spend hours inside a single bash session; its tool outputs (long stdout, large directory listings) would otherwise dominate every subsequent prompt. ARI bounds them with _BoundedOutputTool (ari-skill-paper-re/src/_replicator_agent.py:243), which wraps the inner tool's execute() and post-processes the output through _truncate_tool_output (line 218) at a configurable byte limit. This is the difference between PaperBench upstream's free-form transcript and ARI's bounded, replayable one; it is also why the prompt budget remains predictable across multi-hour rollouts.

5 Evaluation (deferred)

This chapter is intentionally short. The evaluation has not been carried out for this snapshot. The previous draft of this report quoted concrete medians and per-leaf scores; on review we found those numbers were illustrative examples written during R3a drafting, not measurements from a frozen ARI checkpoint, so they have been removed. Only artefacts that are real are kept.

What *is* real:

- The cost-accounting machinery: CostTracker (cost_tracker.py:77) attaches per-call dollar costs to every model invocation; init on line 181 wires the tracker to a checkpoint directory; the resulting cost_usd field is written into results.json at end of run.
- The exploration loop terminates via the stagnation detector (section 3.4); the empirical frequency at which stagnation rather than $T_{\max}$ / $C_{\max}$ fires is intended to be reported here once a snapshot exists.
- The PGF figure scripts (shared/figures/scripts/) are seed-pinned; running makefigures regenerates deterministic synthetic placeholders that pass check_figures, but those placeholders are *not* cited as evidence in the body.

What is *not yet* done:

- Pinning a frozen ARI v0.7.1 checkpoint and extracting tree.json / results.json into shared/figures/data/. Today the data directory holds only *.meta.yaml files marked source.synthetic:true.
- Reporting median run cost, per-seed exploration curves, and per-category rubric distributions from

real reviews. These will appear here in a follow-up PR that lands the frozen checkpoint and re-derives the aggregates.

- Reporting any PaperBench-style replication number from the `paper-re` skill on real target papers.

Until those follow-ups land, this chapter contains no quantitative claims. Cross-references from other chapters (e.g. in section 3.4) point here only to mark where the empirical evidence will live, not where it currently lives.

6 Related Work

We position ARI against four threads of prior art.

6.1 Tree search for code

AIDE [2] formalises ML-engineering as a tree search over plan / code / debug states; AlphaCode [3] is the canonical reference for large-scale generate-and-filter on competitive programming; the MLE-bench evaluator [1] is now the standard for measuring ML-engineering agents. ARI inherits the BFTS skeleton from AIDE but adds (i) an LLM-driven candidate selector (section 3.2), which avoids fixing a closed-form scalar utility and lets the selection prompt weigh `has_real_data`, full `metrics`, depth, retry count, and a soft `diversity_bonus` jointly, and (ii) the cross-run lineage (section 3.5) which neither AIDE nor MLE-bench formalises.

6.2 Autonomous research agents

The AI Scientist family [4] closes a similar loop end-to-end (idea → experiment → paper) but does not expose the rubric and refine cycle as a first-class object. VirSci [9] simulates several scientist personas and uses their consensus to score outputs; Agent Laboratory [6] treats the agent as a collaborator rather than a driver. ARI differs in two operational respects: **everything lives in a checkpoint** (P1 / P4), and the rubric is the contract between explore and write.

6.3 Paper evaluation

PaperBench [8] is the closest analogue to our `paper-re` integration; it pioneered the rubric-as-tree formalism. We adopt the same per-leaf grade and aggregate weight (eq. (3)) but extend it with a per-axis floor used as a refine guard (section 4.5).

6.4 Foundations

The agent loop follows the ReAct pattern [11]; the refine cycle is descended from Self-Refine [5] and Reflexion [7]; and the choice to give the LLM a step-by-step scratchpad in *Think* is descended from Chain-of-Thought [10]. None of these references propose a checkpoint-scoped state; that is ARI's contribution.

An earlier draft cited a separate ML-research-agent benchmark, but we removed that reference once it became clear that none of the candidate sources met our verification rule (section 4.5); we keep the related-work coverage to entries that can be S2-verified.

7 Discussion and Limitations

7.1 Determinism vs novelty

P2, the determinism principle, is the binding constraint of the system. Strictly enforcing P2 forces *ari-skill-memory* to declare "no LLM calls" — it must score its retrievals by a deterministic keyword function rather than an embedding model whose output depends on a remote service. The trade-off is that retrieval quality is bounded by what keyword scoring can recover.

This is not an accident: every reproducibility win comes at the cost of a closed door for non-determinism. We accept the cost because the checkpoint, not the run, is the contract with downstream users.

7.2 Scaling limits

The single-machine assumption underlying the BFTS scheduler is the biggest scaling constraint. A run that wants to expand $b > 10$ in parallel needs a different scheduler; the current scheduler at `ari-core/ari/orchestrator/scheduler.py` serialises expansions of the same parent. A multi-host expansion would require extending the lineage discipline (P4) to handle merges, not just ancestor walks.

A separate constraint is the cost of the LLM grader. Empirically the grader is one of the dominant line items in a run, alongside the explorer; whether it is the second-largest in absolute dollars is something section 5 is meant to settle once a frozen checkpoint exists. Caching grader outputs by (paper-hash, rubric-hash) would help; we have not yet shipped this.

7.3 Open problems

1. **Off-policy refine**. The current refine cycle uses the same model as the writer. Decoupling the two so the writer and the refiner can be different models would let cheaper models do the heavy structural rewrites.

2. **Rubric inheritance across checkpoints**. A child checkpoint should be able to reuse its parent's rubric by default; right now the rubric is regenerated.

3. **Tool-failure recovery**. A failed tool call currently marks the node failed. A recovery policy that retries with a degraded tool (e.g. a smaller model) would reduce the long-tail cost.

These three are tractable; we expect each to be a single-PR change landed before v0.8.

A Notation Table

The table below mirrors the macros defined in `shared/notation.tex`. Every symbol used in the body must appear here; a static check (`check_notation.py`) verifies that no body symbol is undefined.

Symbol	Meaning
$\mathcal{N}$	set of nodes in the search tree
$\mathcal{T}$	search tree $(\mathcal{N}, E)$
$n \in \mathcal{N}$	individual node
$\mathrm{ch}(n)$	children of n
$\mathrm{pa}(n)$	parent of n
$\mathrm{depth}(n)$	depth of n
$s(n)$	node state $\in$ {open, eval, done, failed}
$r(n)$	scalar reward of n
$\mathbf{e}(n)$	per-axis evaluation vector
ϕ	axis $\to$ scalar aggregator
$\mathrm{div}(n)$	diversity bonus (eq. (1))
$\_\mathrm{fallback}\_\mathrm{score}(n)$	deterministic fallback ranking (eq. (2))
$T_{\max}, C_{\max}$	step / cost budgets
R	rubric tree
c_i	rubric leaf i's category
w_i	rubric leaf i's weight
judge_i	rubric leaf i's judge
P	paper draft
$\mathrm{score}(P)$	paper aggregate score (eq. (3))
Refine	refine operator (eq. (4))
ckpt	checkpoint directory
$\mathrm{lin}(n)$	lineage chain ending at n

References

[1] Jun Shern Chan et al. *MLE-bench: Evaluating Machine Learning Agents on Machine Learning Engineering*. 2024. DOI: 10.48550/arXiv.2410.07095. arXiv: 2410.07095 [cs.LG]. URL: https://arxiv.org/abs/2410.07095.

[2] Zhengyao Jiang et al. *AIDE: AI-Driven Exploration in the Space of Code*. 2025. arXiv: 2502.13138 [cs.AI]. URL: https://arxiv.org/abs/2502.13138.

[3] Yujia Li et al. *Competition-Level Code Generation with AlphaCode*. 2022. DOI: 10.1126/science.abq1158. arXiv: 2203.07814 [cs.PL]. URL: https://arxiv.org/abs/2203.07814.

[4] Chris Lu, Cong Lu, Robert Tjarko Lange, Jakob Foerster, Jeff Clune, and David Ha. *The AI Scientist: Towards Fully Automated Open-Ended Scientific Discovery*. 2024. arXiv: 2408.06292 [cs.AI]. URL: https://arxiv.org/abs/2408.06292.

[5] Aman Madaan et al. *Self-Refine: Iterative Refinement with Self-Feedback*. 2023. DOI: 10.48550/arXiv.2303.17651. arXiv: 2303.17651 [cs.CL]. URL: https://arxiv.org/abs/2303.17651.

[6] Samuel Schmidgall et al. *Agent Laboratory: Using LLM Agents as Research Assistants*. 2025. DOI: 10.18653/v1/2025.findings-emnlp.320. arXiv: 2501.04227 [cs.AI]. URL: https://arxiv.org/abs/2501.04227.

[7] Noah Shinn, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. *Reflexion: Language Agents with Verbal Reinforcement Learning*. 2023. arXiv: 2303.11366 [cs.AI]. URL: https://arxiv.org/abs/2303.11366.

[8] Giulio Starace et al. *PaperBench: Evaluating AI's Ability to Replicate AI Research*. 2025. DOI: 10.48550/arXiv.2504.01848. arXiv: 2504.01848 [cs.AI]. URL: https://arxiv.org/abs/2504.01848.

[9] Haoyang Su et al. *Many Heads Are Better Than One: Improved Scientific Idea Generation by a LLM-Based Multi-Agent System*. 2024. DOI: 10.18653/v1/2025.acl-long.1368. arXiv: 2410.09403 [cs.CL]. URL: https://arxiv.org/abs/2410.09403.

[10] Jason Wei et al. *Chain-of-Thought Prompting Elicits Reasoning in Large Language Models*. 2022. arXiv: 2201.11903 [cs.CL]. URL: https://arxiv.org/abs/2201.11903.

[11] Shunyu Yao et al. *ReAct: Synergizing Reasoning and Acting in Language Models*. 2023. arXiv: 2210.03629 [cs.CL]. URL: https://arxiv.org/abs/2210.03629.